

Definiera egna datatyper

I C++ kan man definiera helt egna datatyper om man vill. Detta görs ju redan implicit då man skapar sammansatta datatyper eller använder enumereringar. Förutom dessa kan man definiera egna typer som har en existerande datatyp som grund. Detta görs med hjälp av nyckelordet `typedef`. Allmänna formen för en typdefinition är:

```
typedef ursprungsdatatyp ny_datatyp;
```

Man ger i princip en existerande datatyp ett nytt namn. Den nya datatypen fungerar exakt som den gamla typen med avseende på aritmetik, tilldelningar o.s.v. Några exempel på nya definierade typer:

```
typedef long ErrorCode;  
typedef unsigned char uchar;
```

Via den första definitionen har vi en ny datatyp som heter `ErrorCode` som fungerar precis som en `int`. Den kan göra ett program lättare att läsa då man enkelt förstår vad en funktion returnerar om den returnerar en `ErrorCode`. Det andra fallet definierar en ny typ `uchar` som en ersättare för `unsigned char`. Det är kortare att skriva `uchar`, och om man använder dylika parametrar på många ställen kanske det är mera överskådligt att använda sig av `uchar`. Här märker vi att endast det sista ordet i en `typedef` blir den nya datatypen, medan allt mellan det ordet och `typedef` anses tillhöra namnet på originaltypen. Det är alltså tillåtet att ha en ursprungsdatatyp med flera än ett ord i sin definition, t.ex. en `struct`-definition.

I de flesta fall kan man använda en enumerering eller en klass istället för `typedef`, så det är ganska sällan den används i praktiken. I vissa fall försvarar den dock sin plats i C++:s vokabulär.